

Nombre: José Antonio Canto Marcos

Fecha: 20/06/26
Codigo: 16200113

Examen de Desarrollo de Sistemas Móviles

Análisis de Arquitectura y Concurrency

1) El patrón repositorio como SSOT

- Se le considera como única fuente de verdad porque centraliza todo el acceso de los datos a la aplicación ya que todas las solicitudes pasan por el repositorio siendo el único responsable decidiendo que entregar y como obtenerlo, actuando de fachada por ocultar la complejidad de las distintas fuentes de datos a través de una interfaz.

La coordinación entre el room y la api remota sigue un flujo como el de la UI solicitando datos a la ViewModel, esta lo solicita al repositorio el cual consulta en la base de datos local mediante el Room en caso de no

}

3) Flujo reactivo (Cold vs Hot)

- State Flow es más adecuado para la gestión de estados, ya que implementa un flujo caliente permitiendo que múltiples observadores accedan de manera inmediata y consistente al estado actual de la aplicación a través del Flow que reinicia su ejecución para cada lector y no conserva emisiones previas.

Se prefiere State Flow por garantizar propagación y persistencia continua del estado por ello se utiliza como mecanismo principal para exponer ~~datos~~ estados desde el ViewModel hacia la UI de manera más eficiente.

4) Inyección de Dependencias (DI) manual

persistencia continuada del estado por ello se utiliza como mecanismo principal para exponer ~~los~~ estados desde el ViewModel hacia la UI de manera mas eficiente

4) Inyección de Dependencias (DI) Manual

- La DI Manual mediante la clase Application centraliza la creación y provisión de dependencias, desacoplando los componentes de la lógica de instanciación y facilitando la reutilización de objetos compartidos.

El lazy implementa una inicialización diferida y segura permitiendo crear dependencias de infraestructura únicamente cuando son requeridas, optimizando el consumo de memoria.

Sección de Completer

- 1) El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow

La coordinación entre el Room y la API remota sigue un flujo como el de la UI solicitando datos a la ViewModel, este lo solicita al repositorio el cual consulta en la base de datos local mediante el Room en caso de no existir, estos desactualizados al repositorio realiza la petición a la API el cual los datos obtenidos son almacenados en la Room notificando los cambios mediante Flow esto al ViewModel lo recibe y la UI se actualiza.

2) Corrutinas y Structured concurrency

- Es un alcance vinculado al ciclo de vida de un ViewModel, todas las corrutinas lanzadas en este scope se ejecutaran mientras exista el ViewModel y así evitar tareas en segundo plano se sigan ejecutando ocasionando consumo de recursos y fugas de memoria.

En el caso de una pantalla cerrada antes de recibir respuesta con este scope se hizo de nuevo ViewModel cancelandolo y todas las corrutinas hijos asociadas tambien son canceladas por lo que ya no se actualiza la UI.

La DZ Manual mediante la clase AppDomain crea la creación y provisión de dependencias, desacopladas los componentes de la lógica de inicialización y facilitando la reutilización de objetos compartidos.

Por lo que implementa una inicialización diferida y segura permitiendo crear dependencias de infraestructura únicamente cuando son requeridas, optimizando el consumo de memoria.

2) Sección de completos

- 1) El objeto de datos que emite automáticamente cada vez que una tarea de Room Lambda se implementa usando el tipo Flow
- 2) El operador Dispatcher permite ejecutar tareas pesadas en el main thread específicamente optimizado por CPU como el parser main de datos

- 3) Para que un servicio en primer plano de ubicación sea realizado en Android 10+, debe declararse en el manifiesto con el tipo location
- 4) La función Container de la clase Application es la que garantiza que los componentes comparten la misma instancia en el repositorio mediante un cast
- 5) El componente que permite transformar una API legacy en callbacks antiguos en una función susceptible moderna se llama callbackFlow
- 6) El archivo de metadata que describe la estructura de la app al sistema operativo se llama AndroidManifest.xml
- 7) En la arquitectura AOIP la capa que actúa como puente entre el framework y los controladores se denomina HAL (Hardware Abstraction Layer)

se denomina HAL (Hardware Abstraction Layer)

- 8) El Operador Flow que permite combinar múltiples fuentes (como GPS y sensores) para mejorar la exactitud se llama combine
- 3) Desarrollo de código y login personalizado
 - 1) Repositorio y DT

```

class UsuarioRepository {
    private val usuariosDao: UsuariosDao
}

class MiApplication: Application() {
    private var database by lazy {
        AppDatabase.getDatabase(this)
    }
    val usuarioRepository by lazy {
        UsuarioRepository(database, usuariosDao)
    }
}
          
```

2) a) data class Usuario(
 val nombre: String,
 val apellido: String,
 val edad: Int?)
)

b) objeto con datos reales

val usuario = Usuario(
 nombre = "Tox",
 apellido = "Conto rois",
 edad = 26
)

c) Calculo de Puntaje de calificación

val puntaje = (usuario.nombre.length) + (usuario.apellido.length)

d) imprimir mensaje

~~println("Tox")~~

println(" { usuario.nombre } { usuario.apellido }
 tiene un puntaje de calificación de \$ puntaje")

c) Calculo de Puntaje de calificación

val puntaje = (usuario.nombre.length) + (usuario.apellido.length)

d) imprimir mensaje

~~println("Tox")~~

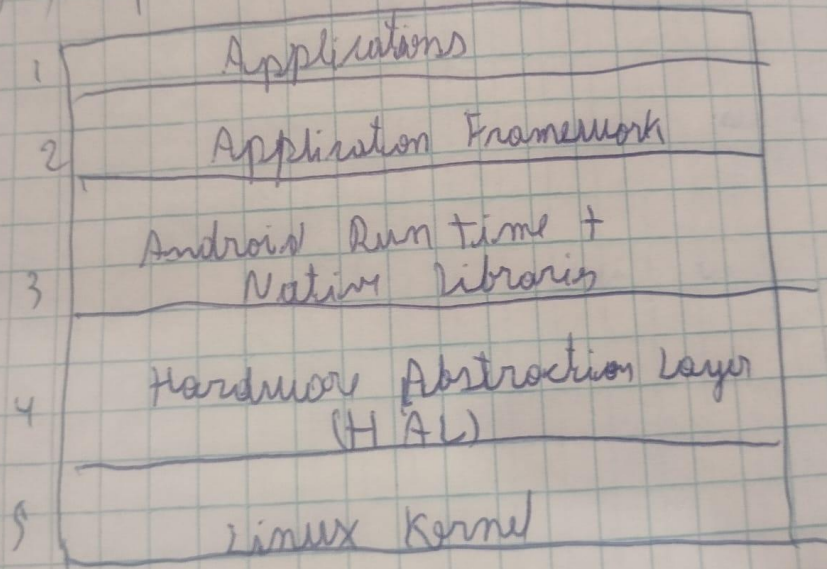
println(" { usuario.nombre } { usuario.apellido }
 tiene un puntaje de calificación de \$ puntaje")

e) uso de operador Elvis (?)

~~val puntaje = usuario.nombre?.length + usuario.apellido?.length ?: 0~~

val puntaje = usuario.nombre.length +
 (usuario.apellido.length ?: 0)

4) 1) Architecture AOSP



2) Identificación

- 5) Linux Kernel
- 2) Identificación
1. Applications: capa donde residen las aplicaciones instaladas por el usuario y aplicaciones del sistema
 2. Application Framework: ~~Proporcionamos~~ Proporcionamos las APIs y servicios que utilizan las aplicaciones para interactuar con el sistema operativo
 3. Android Run time + Native Libraries: Capa encargada de ejecutar las aplicaciones Android

Componentes: ART: Entorno de ejecución de aplicaciones Android

Core Libraries: Bibliotecas principales que implementan gran parte de APIs estándar de Java/Kotlin
 4. Hardware Abstraction Layer (HAL): Capa de Abstracción entre el framework Android y los controladores del hardware como cámaras, GPS, Wi-Fi, etc.
 - 5) Linux Kernel: Capa más baja, gestiona los recursos del hardware y del sistema.